Homework 7

1) Write a C program to use Timer0 and Timer2 to generate 1 kHz and 4 kHz square waves on PORTB bit 1 and PORTB bit 2, respectively, using interrupts while at the same time. Assume Fosc = 8 MHz.

T1 = 1/f1 = 1/1000 = 0.001 = 1000 us

T1half = 500 us

N1 = tdelay/tMC => 500 10^-6 s/[(1cc/1MC)(1/8 10^6 cyc/s)] = 4000MC

4000/64 = 62.5 => choose 63 as preload with prescale of 64

T2 = 1/f2 = 1/4000 = 0.0025 = 250 us

T2half = 125 us

N2 = tdelay/tMC => 125 10^-6 s/[(1cc/1MC)(1/8 10^6 cyc/s)] = 1000 MC

1000/8 = 125 => choose 125 as preload with prescale of 8

```
#include <avr/io.h>
#include <avr/interrupt.h>


ISR(TIMER0_OVF_vect);
ISR(TIMER2_OVF_vect);

int main(void)
{

        DDRB |= 0x06; // 0b00000110 - PB1(1kHz) and PB2(4kHz)
        DDRC = 0x00;  // PORTC INPUT
        PORTC = 0xff; // pullup PORTC
        DDRD = 0xff;  // PORTD OUTPUT

        TCNT0 = -63;
        TCNT2 = -125;

        TCCR0 = 0x03;  // TIMER0 - prescale 64
        TCCR2 = 0x02;  // TIMER2 - prescale 8

        TIMSK = 0x41  ; // 0b01000001 - enable Timer0 and Timer2 overflow interrupts
        sei();

    while (1)
    {
                PORTD = PINC;
```

```
    }
        return 0;
}

ISR(TIMER0_OVF_vect)
{
        PORTB ^= 0x02;  // toggle PB1
        TCNT0 = -63;
}

ISR(TIMER2_OVF_vect)
{
        PORTB ^= 0x04; // toggle PB2
        TCNT2 = -125;
}
```

2) Given the following C program to generate/output a 1 kHz square wave on PB3 and at the same time transfer data from PORTD to PORTA. Convert the use of Timer 0 for generating the square wave which uses polling (below) to the use of Timer 0 with a Timer 0 overflow (TOV0) interrupt. This will require you to: a) configure Timer 0 overflow (TOV0) as an interrupt source, b) write the Interrupt Service Routine (ISR) for Timer 0 overflow (TOV0).

```
// generate a 1 kHz square wave on PB3 and at the same transferring data from PORTD to
PORTA

// TIMER0 FOR SQUARE WAVE
// PORTD => INPUT, PORTB => OUTPUT
//
// FOSC = 8 MHZ
// THALF = 0.5 X 1000 US = 500 US

// n = 500 us/[(1cc/1MC)(1s/8 10^6 cc)] = 4000 MCs

#include <avr/io.h>



int main(void)
{
        DDRB = 0xff;  // PORTB output
        DDRD = 0x00;  // PORTD INPUT
        PORTD = 0xff; // PULLUP RESISTORS FOR PORTD
        DDRA = 0xff;  // PORTA output

        TCCR0 = 0x04; 0b00000100 for prescale of 256 for clk for ATmega32
        TCNT0 = -16;



        while(1)
        {
```

```
		// use TIFR for ATmega32
		while (!(TIFR & (1<<TOV0)))
		{
			PORTA = PIND;
		}

		TCCR0 = 0x00; // turn off timer 0 for reset

		TIFR = 0x01;   // write a 1 to TOV0 to reset overflow flag
		TCNT0 = -16;    // reset preload
		PORTB ^= 0x08; // 00001000
		TCCR0 = 0x04; // turn timer 0 back on

	}

	return 0;
}
```

Solution to Problem 2

```
// generate a 1 kHz square wave on PB3 and at the same transferring data from PORTD to
PORTA

// TIMER0 FOR SQUARE WAVE
// PORTD => INPUT, PORTB => OUTPUT
//
// FOSC = 8 MHZ
// THALF = 0.5 X 1000 US = 500 US

// n = 500 us/[(1cc/1MC)(1s/8 10^6 cc)] = 4000 MCs

#include <avr/io.h>
#include <avr/interrupt.h>
ISR(timer0_ovr_vect);


int main(void)
{
	DDRB = 0xff;  // PORTB output
	DDRD = 0x00;  // PORTD INPUT
	PORTD = 0xff; // PULLUP RESISTORS FOR PORTD
	DDRA = 0xff;  // PORTA output

	TIMSK = 0b00000001; // enable TOIE0
	TCNT0 = -16; // preload for approx 4000 MCs with 256 prescaler
	TCCR0 = 0x04; 0b00000100 for prescale of 256 for clk for ATmega32, turn on timer 0
	sei(); // I = 1 so timer 0 is fully enabled


	while(1)
	{
		PORTA = PIND; // copy received value from PORTD to PORTA
	}
	return 0;
}
```

```
ISR(timer0_ovr_vect)
{
        TCNT0 = -16; // reset preload for next half period
        PORTB ^= 0x08; // 00001000 – toggle PB3 for waveform
}   // no need to turn off timer 0 (RETI from assembly code translation resets TOV0)
    // timer 0 was turned off in polling for resetting TOV0
```

3) Assume Timer0 is programmed in normal mode, no prescaler (TCCR0 = 0x01), TCNT0 = 0xF1, I = 1 in SREG, and TOIE0 = 1. Give the steps for how the Timer0 overflow is triggered and how the Timer0 overflow interrupt is serviced (only need to include steps that get to the Timer0 Interrupt Service Routine). What is the last (assembly) instruction of the Timer0 Interrupt Service Routine and what does this instruction do to restore the program to where it was before the Timer0 interrupt?

Steps for how Timer0 overflow is triggered and serviced:

1) 0xF1 = 241 decimal => 256-241 = 15 ticks to trigger TOV0 =1 (timer 0 overflow)
2) When TOV0 becomes 1, the current program instruction completes execution.
3) The PC (address to the next instruction where the program is currently executing) is pushed onto the stack.
4) The PC is loaded with the vector address of the timer 0 overflow interrupt service routine (0x0016).
5) I is cleared (I = 0) to disable other interrupt sources while the Timer0 interrupt service routine is executing.
6) Program execution continues with the first instruction of the interrupt service routine.

Last instruction of the Timer0 Interrupt Service Routine => RETI (assembly translation). This instruction clears the TOV0 flag, resets I (I = 1) to reenable interrupt sources, and the PC is popped from the stack for the program to resume execution from before the interrupt activation.

4) Assume that Timer2 (TOV2) and Timer0 (TOV0) overflow interrupts and External Interrupt 1 are enabled. What happens when TOV2 and TOV0 occur simultaneously? What happens if all three interrupts occur simultaneously? What determines which interrupt is serviced first?

Interrupt vector addresses:

External interrupt 1 (INT1) 0x0004

Timer 2 overflow 0x000A

Timer 0 overflow 0x0016

The interrupt polling order starts from 0x0000 (reset) and goes to the highest interrupt source vector address (ascending order) to determine which interrupt occurs first if multiple interrupts are activated simultaneously.

If TOV2 and TOV0 occur simultaneously, timer 2 overflow is serviced first since it has a lower vector address. Then, timer 0 overflow is serviced.

If all three interrupts occur simultaneously, the order of interrupt service is:

1) External interrupt 1 (INT1) 0x0004
2) Timer 2 overflow 0x000A
3) Timer 0 overflow 0x0016

5) Convert the assembly program (below) to **your own** C code (ATmega32).

```
.ORG 0
RJMP MAIN

.ORG 0X0016
RJMP TIMER0_OVR_ISR

.ORG 50
LDI R16, 0X01
IN R17, DDRB
OR R17,R16
OUT DDRB, R17
LDI R16, 0XFF
OUT DDRC, R16
LDI R16, 0X00
OUT DDRD, R16
LDI R16, 0XFF
OUT PORTD, R16

LDI R16, -20
OUT TCNT0, R16
LDI R16, 0X02
OUT TCCR0, R16
LDI R16, 0X01
OUT TIMSK, R16
SEI
HERE: IN R18, PIND
OUT PORTC, R18
RJMP HERE
```

```
#include <avr/io.h>
#include <avr/interrupt.h>
ISR(timer0_ovr_vect);
int main() {
        DDRB |= 0X01;
        DDRC = 0XFF;
        DDRD = 0X00;
        PORTD = 0XFF;
        TCNT0 = -20;
        TCCR0 = 0X02;
        TIMSK = 0X01;
        SEI();
        while(1) {
                PORTC = PIND;
        }
        return 0;
}

ISR(timer0_ovr_vect) {
        PORTB ^= 0X01;
        TCNT0 = -20
}
```

```
.ORG 100
TIMER0_OVR_ISR:  LDI R19, 0X01
IN R20, PORTB
EOR R20,R19
OUT PORTB, R20
LDI R16, -20
OUT TCNT0, R16
RETI
```

6) Given a simple security system with a door sensor and window sensor that are connected to the ATmega32 to PD2 (INT0) and PD3 (INT1), respectively. Using external interrupts for the door and window sensors, let the door sensor be falling edge triggered for an open door event and the window sensor be rising edge triggered for an open window event. For an open door event, turn on an LED at PD7 for a short duration and then turn off the LED, and for an open window event, turn on an LED at PD6 for a short duration and then turn off the LED. Assume the LEDs are common cathode (1 turns on an LED). Let the short duration be given as an assumed function DELAY() (just call DELAY() for the short duration.

```
#include <avr/io.h>
#include <avr/interrupt.h>

ISR(INT0_vect);
ISR(INT1_vect);
void DELAY();

int main(void)
{
   DDRD &= 0b11110011; // PD2 (INT0) door sensor, PD3 (INT1) window sensor
   DDRD |= 0b11000000; // PD6, PD7 OUTPUTS FOR LEDS
   PORTD |= 0b00001100; // pullup resistor for PD2 and PD3

   MCUCR = 0b00001110; // bits 3 2 = 11 for rising edge for window sensor, bits 1 0 = 10
for falling edge for door sensor

   GICR = (1<<INT1) | (1<<INT0);


   while(1);  // infinite loop letting ATmega32 monitor status of door sensors
}

ISR(INT0_vect)  // falling edge door opening event
{
       PORTD |= 0b10000000;  // turn on LED at PD7 for a short duration
       DELAY();
       PORTD &= 01111111; // turn off LED at PD7
}

ISR(INT1_vect)  // rising edge window opening event
{
       PORTD |= 0b01000000;  // turn on LED at PD6 for a short duration
```

```
        DELAY();
        PORTD &= 10111111; // turn off LED at PD6
}

void DELAY()
{
        int i,j;

        for (i=0; i<500; i++)
                for (j=0; j<500; j++);
}
```

7) Chapter 10: 1, 3, 25, 31, 32

1) Interrupt

3) $00, $28

25) False. The CPU automatically clears the TOV0 flag.

31) $02, $04, and $06 respectively – PD2, PD3, and PB2 respectively.

32) GICR

```
LDI R16, 1<<INT0
OUT GICR, R16
```

Problems are below.

SECTION 10.1: AVR INTERRUPTS

1. Which technique, interrupt or polling, avoids tying down the microcontroller?
2. List some of the interrupt sources in the AVR.
3. In the ATmega32 what memory area is assigned to the interrupt vector table?
4. True or false. The AVR programmer cannot change the memory address location assigned to the interrupt vector table.
5. What memory address is assigned to the Timer0 overflow interrupt in the interrupt vector table?
6. What memory address is assigned to the Timer1 overflow interrupt in the interrupt vector table?
7. Do we have a memory address assigned to the Time0 compare match interrupt in the interrupt vector table?
8. Do we have a memory address assigned to the external INT0 interrupt in the interrupt vector table?
9. To which register does the I bit belong?
10. Why do we put a JMP instruction at address 0?
11. What is the state of the I bit upon power-on reset, and what does it mean?
12. Show the instruction to enable the Timer0 compare match interrupt.
13. Show the instruction to enable the Timer1 overflow interrupt.
14. The TOIE0 bit belongs to register_____.
15. True or false. The TIMSK register is not a bit-addressable register.
16. With a single instruction, show how to disable all the interrupts.
17. Show how to disable the INT0 interrupt.
18. True or false. Upon reset, all interrupts are enabled by the AVR.
19. In the AVR, how many bytes of program memory are assigned to the reset?

8)

SECTION 10.2: PROGRAMMING TIMER INTERRUPTS

20. True or false. For each of Timer0 and Timer1, there is a unique address in the interrupt vector table.

21. What address in the interrupt vector table is assigned to Timer2 overflow?
22. Show how to enable the Timer2 overflow interrupt.
23. Which bit of TIMSK belongs to the Timer0 overflow interrupt? Show how it is enabled.
24. Assume that Timer0 is programmed in Normal mode, TCNT0 = $E0, and the TOIE0 bit is enabled. Explain how the interrupt for the timer works.
25. True or false. The last three instructions of the ISR for Timer0 are:

```
LDI   R20,0x01
OUT   TIFR,R20    ;clear TOV0 flag
RETI
```

26. Assume that Timer1 is programmed for CTC mode, TCNT1H = $01, TCNT1L = $00, OCR1AH = $01, OCR1AL = $F5, and the OCIE1A bit is enabled. Explain how the interrupt is activated.
27. Assume that Timer1 is programmed for Normal mode, TCNT1H = $FF, TCNT1L = $E8, and the TOIE1 bit is enabled. Explain how the interrupt is activated.
28. Write a program using the Timer1 interrupt to create a square wave of 1 Hz on pin PB7 while sending data from PORTC to PORTD. Assume XTAL = 8 MHz.
29. Write a program using the Timer0 interrupt to create a square wave of 3 kHz on pin PB7 while sending data from PORTC to PORTD. Assume XTAL = 1 MHz.

9)

SECTION 10.4: INTERRUPT PRIORITY IN THE AVR

47. Explain what happens if both INT1F and INT2F are activated at the same time.
48. Assume that the Timer1 and Timer0 overflow interrupts are both enabled. Explain what happens if both TOV1 and TOV0 are activated at the same time.
49. Explain what happens if an interrupt is activated while the AVR is serving an interrupt.
50. True or false. In the AVR, an interrupt inside an interrupt is not allowed.

10)

SECTION 10.3: PROGRAMMING EXTERNAL HARDWARE INTERRUPTS

30. True or false. An address location is assigned to each of the external hardware interrupts INT0, INT1, and INT2.
31. What address in the interrupt vector table is assigned to INT0, INT1 and INT2? How about the pins?
32. To which register does the INT0 bit belong? Show how it is enabled.
33. To which register does the INT1 bit belong? Show how it is enabled.
34. Show how to enable all three external hardware interrupts.
35. Assume that the INT0 bit for external hardware interrupt is enabled and is negative edge-triggered. When is the interrupt activated? How does this interrupt work when it is activated.
36. True or false. Upon reset, all the external hardware interrupts are negative edge triggered.
37. The INTF0 bit belongs to the _____ register.
38. The INTF1 bit belongs to the _____ register.
39. Explain the role of INTF0 and INT0 in the execution of external interrupt 0.
40. Explain the role of I in the execution of external interrupts.
41. True or false. Upon power-on reset, all of INT0–INT2 are positive edge triggered.
42. Explain the difference between low-level and falling edge-triggered interrupts.
43. Show how to make the external INT0 negative edge triggered.
44. True or false. INT0–INT2 must be configured as an input pin for a hardware interrupt to come in.
45. Assume that the INT0 pin is connected to a switch. Write a program in which, whenever it goes low, the content of PORTC increases by one.